Implementation of Dijkstra's Single-Source Shortest Path Algorithm

Verified by AI Lab Agent | Powered by Google Gemma Open-Weight Model

[VERIFIED: PASSED] Subprocess Sandbox 5.0s Safeguard | Runtime: 1243.06 ms | Exit Code: 0

Student Name	Aryan Rayani	**Enrollment ID**	2026-ENG-CS-042
Academic Course	CS302 - Design and Analysis of	**Target Source File**	sample_code.py

1. Experiment Objective & Scope

To formulate, implement, and verify Dijkstra's greedy algorithm using a binary min-heap priority queue for computing single-source shortest path trees on weighted directed/undirected graphs. The experiment validates algorithmic correctness, edge relaxation invariants, and optimal substructure properties.

Problem Definition & Computational Invariants:
Given a directed graph G = (V, E) with non-negative edge weights w: E -> R+ and a source vertex s in V, determine the shortest path distance delta(s, v) and reconstruct the shortest path trajectory from s to every reachable vertex v in V.

2. Algorithm Theory & Code Mechanics

Dijkstra's algorithm is grounded in the Greedy Choice Property and Optimal Substructure theorem. It maintains two disjoint sets of vertices: S (vertices whose shortest distance from source is finalized) and V \ S (vertices in the candidate priority queue). At each step, the algorithm greedily extracts the vertex u in V \ S with minimum tentative distance d[u]. For every adjacent edge (u, v) with weight w(u, v), the relaxation step tests whether d[u] + w(u, v) < d[v]. If true, d[v] is updated to d[u] + w(u, v) and its predecessor pi[v] is recorded as u. Because edge weights are strictly non-negative, the greedy choice is globally optimal (No subsequent relaxation can reduce d[u]).

Step-by-Step Execution Mechanics:
1. Distance Initialization: All vertices are assigned tentative distance infinity, with the designated source vertex initialized to 0.0.
2. Min-Heap Maintenance: Python's standard `heapq` module stores pairs of `(cumulative_distance, vertex_id)`, providing O(log V) extract-min operations.
3. Vertex Finalization Loop: Vertices are popped from the min-heap. A `visited` hash-set safeguards against re-processing stale entries.
4. Edge Relaxation & Predecessor Tracking: For each outbound edge from the current vertex, edge costs are evaluated; shorter paths trigger updates and push new candidate pairs onto the priority queue.
5. Backtracking Path Reconstruction: The `reconstruct_path` function starts from target destination vertex 'Z' and backtracks through the predecessor dictionary to produce the exact sequence of router hops.

3. Verified Student Source Code

Listing verified in local execution sandbox (sample_code.py)

```
 1 | """
 2 | Experiment 04: Implementation of Dijkstra's Single-Source Shortest Path Algorithm
 3 | Student Name: Aryan Rayani
 4 | Course: CS302 - Design and Analysis of Algorithms (DAA) Lab
 5 | Enrollment No: 2026-ENG-CS-042
 6 | """
 7 |
 8 | import heapq
 9 | import time
10 | from typing import Dict, List, Tuple, Optional
11 |
12 | def dijkstra(graph: Dict[str, List[Tuple[str, int]]], start_node: str) -> Tuple[Dict[s...
```

```
    """
    Computes shortest paths from start_node to all reachable nodes using a min-heap pr...

    Time Complexity: O((V + E) log V)
    Space Complexity: O(V + E)
    """
    # Initialize distances with infinity
    distances: Dict[str, float] = {node: float('inf') for node in graph}
    distances[start_node] = 0.0

    # Predecessor map for path reconstruction
    predecessors: Dict[str, Optional[str]] = {node: None for node in graph}

    # Priority queue stores tuples of (distance, node)
    pq: List[Tuple[float, str]] = [(0.0, start_node)]

    visited = set()
    step_counter = 0

    print(f"[DAA LAB] Initializing Dijkstra's search from source node: '{start_node}'")
    print("-" * 65)

    while pq:
        current_distance, current_node = heapq.heappop(pq)

        # If node has already been finalized, skip
        if current_node in visited:
            continue

        visited.add(current_node)
        step_counter += 1
        print(f"Step {step_counter:02d}: Finalized Node '{current_node}' with cumulati...

        # Relax neighboring edges
        for neighbor, weight in graph.get(current_node, []):
            if neighbor in visited:
                continue

```

4. Live Subprocess Terminal Console Output

Captured stdout stream (Exit Code: 0 | Elapsed: 1243.06 ms)

```
================================================================
EXPERIMENT 04: DIJKSTRA'S ALGORITHM VERIFICATION
================================================================
[DAA LAB] Initializing Dijkstra's search from source node: 'A'
-----------------------------------------------------------------
Step 01: Finalized Node 'A' with cumulative cost = 0.0
         Relaxed edge (A -> B): New tentative cost = 4.0
         Relaxed edge (A -> C): New tentative cost = 2.0
Step 02: Finalized Node 'C' with cumulative cost = 2.0
         Relaxed edge (C -> B): New tentative cost = 3.0
         Relaxed edge (C -> D): New tentative cost = 10.0
         Relaxed edge (C -> E): New tentative cost = 12.0
Step 03: Finalized Node 'B' with cumulative cost = 3.0
         Relaxed edge (B -> D): New tentative cost = 8.0
Step 04: Finalized Node 'D' with cumulative cost = 8.0
         Relaxed edge (D -> E): New tentative cost = 10.0
```

```
         Relaxed edge (D -> Z): New tentative cost = 14.0
Step 05: Finalized Node 'E' with cumulative cost = 10.0
         Relaxed edge (E -> Z): New tentative cost = 13.0
Step 06: Finalized Node 'Z' with cumulative cost = 13.0
----------------------------------------------------------------

[RESULT 1: ALL-PAIRS DISTANCES FROM NODE 'A']
  Target: A | Shortest Latency: 0.0 ms
  Target: B | Shortest Latency: 3.0 ms
  Target: C | Shortest Latency: 2.0 ms
  Target: D | Shortest Latency: 8.0 ms
  Target: E | Shortest Latency: 10.0 ms
  Target: Z | Shortest Latency: 13.0 ms

[RESULT 2: OPTIMAL ROUTE TO DESTINATION 'Z']
  Path: A -> C -> B -> D -> E -> Z
  Total Path Cost: 13.0 ms
  Execution Time: 0.0462 ms

[VERIFICATION STATUS] All unit assertions passed successfully! Exit Code: 0
================================================================
```

5. Big-O Asymptotic Complexity Matrix

Dimension	Bound	Mathematical Rigor & Justification
Time (Best Case)	**`O((V + E) log V)`**	O((V + E) log V) - With binary min-heap, each vertex is extracted once a...
Time (Average Case)	**`O((V + E) log V)`**	O((V + E) log V) - Dominant cost stems from E heap push operations and V...
Time (Worst Case)	**`O((V + E) log V)`**	O((V + E) log V) - For dense graphs where E ~= V^2, time complexity uppe...
Auxiliary Memory	**`O(V)`**	O(V) - Storage required for distances dictionary, predecessor map, and v...
Total Space	**`O(V + E)`**	O(V + E) - Total memory required to store the adjacency list representat...

6. Gemma Viva-Voce Defense Preparation

Q1: Why does Dijkstra's algorithm fail or produce incorrect results on graphs with negative edge weights?

Model Answ
permanent
yield a lowe

Q2: What is the theoretical time complexity difference between using an Adjacency Matrix vs. Adjacency List with a Min-Heap?

Model Answ
graphs (E ~
Heap), whic

Q3: Explain the Edge Relaxation condition and how path reconstruction is mathematically achieved.

Model Answ
d[v] is relax
pi[pi[t]], unt

7. Faculty Assessment & Endorsement Block

Evaluation Component	Max Score	Score Awarded	Faculty Feedback
Algorithm Implementation & Robustness	**10**	**[/ 10]**	*Logic conforms to min-heap specifications*

Sandbox 5.0s Verification & Test Harness	**10**	**[/ 10]**	*Assertions verified; zero timeout*
Asymptotic Complexity & Theory	**05**	**[/ 05]**	*Sound mathematical rationale*
Viva-Voce Oral Defense	**05**	**[/ 05]**	*Thorough algorithmic comprehension*

Faculty Evaluator Signature: _____ **Evaluation Date: October 24, 2026**